

ArogyaRakshak — Technical Project Documentation (v3, Consolidated)

B.E. Computer Engineering | PES Modern College of Engineering, Pune | SPPU 2019

Pattern Team Lead: Viraj Jadhao | Team of 5 | Status: Guide-approved (v2), extended this session (v3)

This document supersedes v2 and the standalone v3 addendum — it is the single source of truth going forward. Everything from v2 is preserved; nothing here breaks or contradicts prior decisions. New material is clearly marked NEW.

1. Overview

Patients upload a hospital bill, a prescription, or an insurance rejection letter, and get an instant plain-language read on whether they've been overcharged or denied unfairly — plus a ready-to-file appeal, a list of government schemes they qualify for, and (NEW) a pre-filled insurance claim they can submit themselves. English, Hindi, and Marathi throughout.

Selected from five candidate FYP ideas (ArogyaRakshak, ClimateX, CivicEye AI, CourtPrep AI, TenderGuard AI) on real end-user reach, public-data-only access, and an existing hackathon build to extend rather than a cold start.

2. System Architecture

Four capability areas sit on one shared context layer — **Kadi** — so a single document upload (or, NEW, a single ABDM-consented history pull) can surface insights across every module without the user re-uploading anything.



Any document type or ABDM consent event triggers the same shared extraction step, lands in Kadi's case context, and from there any module can read what's already known and write back what it finds. That's what "linked, not just co-located" means in practice.

3. The Modules

3.1 BillNyay

(formerly *BillGuard* — "Nyay" = justice)

Core flow: Upload a hospital bill → OCR extracts line items → benchmarks against CGHS government rates → flags overcharges → 5-agent pipeline (Document Auditor → Clinical Reviewer → Regulatory Advisor → Appeal Drafter → QA Judge) drafts an IRDAI-compliant appeal letter as a PDF.

Status: carrying forward and refining the existing hackathon build, not rebuilding from scratch.

Hardened per the challenges review (fixes now committed scope, not optional):

- **Document integrity checks** (not authenticity verification — that's structurally impossible without hospital/insurer database access): line-item-to-total arithmetic consistency, date-logic validation (admission before discharge), duplicate-document hash detection across cases.
- **CGHS benchmarking upgraded** to use CGHS's own published city-tier and room-category rate variants instead of one flat table — meaningfully improves accuracy, and is real technical work worth reporting.

- **Language discipline enforced everywhere:** never "overcharged" as a system verdict; always "flags a deviation from the CGHS benchmark of X%." Applies to UI copy, PDF output, and the appeal letter itself.
- **Legal disclaimer embedded inside the generated appeal letter**, not just shown in the UI.
- **QA Judge agent required to cite the specific retrieved source/clause** backing each claim in the drafted appeal; unsupported claims are dropped before the document is finalized — this is the concrete hallucination-mitigation mechanism, not just "we use RAG."
- **Self-report outcome tracker:** user marks what happened after filing (submitted / response received / resolved) — closes the loop as far as is possible without insurer API access, and generates real outcome data for the final evaluation report.

3.2 SchemeSetu

Core flow: User answers a short intake or uploads a bill/prescription → RAG layer over PMJAY (national) and MJPJAY (Maharashtra state) scheme documents → agent reasons through eligibility rules → outputs which scheme(s) apply and a step-by-step claim guide.

Hardened per the challenges review:

- **"Data last verified: [date]" timestamp** displayed on every SchemeSetu output — scheme rules change periodically and this is the honest way to surface that instead of implying permanent currency.
- Recurring **monthly verification task** (owned by Role 5) against PMJAY/MJPJAY portals; any circular updates get logged and re-ingested into the RAG store.
- **UI labeling discipline:** "Estimated eligibility," never "You qualify" — actual approval is a government verification step outside this system's authority. Include a confidence indicator when RAG retrieval is ambiguous.

3.3 DawaCheck

Core flow: Upload a medicine strip photo or prescription → OCR reads drug name/batch → benchmarks MRP against NPPA ceiling prices for Schedule-I formulations (~800-900 price-controlled drugs) → flags overcharging → suggests generic substitutes.

Deliberately scoped to Schedule-I only — the one segment with a clean, finite, government-published ground-truth price. This boundary is a design decision to defend, not a limitation to apologize for; premium/patented/non-scheduled medicines have no defensible benchmark to check against, and expanding coverage there would undermine the module's credibility.

Hardened per the challenges review:

- Generic-alternative suggestions frame as **awareness only** ("a generic equivalent is available at your nearest Jan Aushadhi/PMBJP store"), never a recommendation to

switch — explicit copy directs the user to their doctor/pharmacist before any medication change.

- Evaluation metric corrected: measure "**awareness delivered**" (opt-in: did the user view the suggestion), not "**behavior changed**." The latter was never a fair or achievable success metric for this module.

3.4 DaaviSetu — Claim & Pre-Authorization Generator (NEW)

(*दावी* = claim; working name, team to confirm or fold into BillNyay as a sub-flow instead of a separate module)

Problem it solves: Insurance application/claim paperwork (cashless pre-authorization, reimbursement claims) is tedious in a way distinct from BillNyay's post-treatment dispute flow — this covers the pre/during-treatment claim-filing pain point ("bypassing the hospital's tedious TPA desk process").

Hard constraint: No private insurer exposes a public claim-submission API, and the government's own cross-payer claims exchange (HCX, under ABDM) is still listed as work-in-progress infrastructure — not usable yet. **This module does not and cannot submit claims directly to any insurer.**

What it does do:

- Takes prescription/diagnosis documents (and, once available, ABDM-sourced history — §4.4) already parsed by Kadi's shared extraction agent.
- Auto-fills the cashless pre-authorization or reimbursement claim form in the field structure insurers commonly require (policy number, diagnosis, treatment plan, hospital details, cost estimate) — sourced from publicly available blank claim form templates insurers publish on their own websites.
- Outputs a submission-ready package (PDF + structured summary) for the user to review and submit via whichever channel their insurer requires.

Feasibility: buildable now, within existing architecture — reuses the same agent-pipeline pattern already proven in BillNyay's appeal drafter. No new external data dependency.

3.5 Grievance Tracking & IRDAI Complaint Drafting (NEW — extends BillNyay)

Constraint discovered this session: IRDAI's Bima Bharosa portal has no public API and is login-gated. Critically, IRDAI's own stated policy is that it does **not** entertain complaints filed on a policyholder's behalf by third parties or agents. This rules out both automated status-polling and automated submission — not as a technical gap, but as a compliance boundary.

What's buildable instead:

- **Auto-drafted complaint package**, same 5-agent pipeline BillNyay already uses for appeals, output in Bima Bharosa's required field structure (policy number, insurer,

grievance type, description) with a deep link straight to the portal's "Register New Grievance" page. User pastes/uploads and submits themselves.

- **Self-reported status tracker:** an in-app field the user updates ("Not yet filed / Filed — awaiting response / Resolved / Escalated"), tied to their case in Kadi.
 - **Reminder nudges** based on IRDAI's published turnaround times, so the user knows when a response is due or when escalation becomes possible.
-

4. Kadi — Shared Context Layer

(कड़ी — "link")

Not a fourth module — infrastructure all modules depend on.

4.1 Case & entity model

Table	Purpose
<code>cases</code>	One record per patient session/case — the anchor everything else attaches to
<code>entities</code>	Extracted structured data (medicine, hospital, procedure, amount, category), tagged by type
<code>case_entities</code>	Links entities to the case and the document/module that produced them

Sits alongside each module's own domain tables (CGHS rates, PMJAY/MJPJAY rules, NPPA prices) in the same Postgres instance — not a new database, a new layer in the one already there.

4.2 Shared extraction agent

One extraction agent normalizes any input document (bill, prescription, rejection letter, and NEW: ABDM-pulled records) into a common schema — hospital, diagnosis/procedure, medicines, amount, date, category — rather than each module running its own bespoke parser.

4.3 Entity resolution pipeline

The genuinely hard, genuinely defensible piece — full version, cross-script included, per team decision.

Three similarity signals feed a combined confidence score:

Signal	Catches	Tool
String similarity	Surface variants — "Paracetamol 650mg" vs "Paracetamol"	Edit distance / token overlap
Transliteration matching	Same word, different script — क्रोसिन ↔ Crocin	IndicXlit (AI4Bharat) — <code>pip install ai4bharat-transliteration</code> , MIT-licensed
Semantic/cross- lingual similarity	Translated concepts — बुखार ↔ Fever	IndicSBERT (L3Cube) — <code>l3cube-pune/indic-sentence-similarity-sbert</code>

New entity mention → scored against Kadi (string · transliteration · IndicSBERT) → **High confidence: merge** / **Medium confidence: ask user** / **Low confidence: new entity**.

Both tools are open-source and pretrained — no training pipeline of your own required. Blocking happens within a case (and by entity type) before scoring, so this stays cheap. A small logistic regression combining the three signal scores into one confidence value is a reasonable, legitimate approach — real ML, not overkill.

Free win: the medicine entity-resolution work and DawaCheck's generic-substitute feature need the same brand-name ↔ active-ingredient mapping. Build it once, both features benefit.

Research contribution note: this pipeline is also the basis of the entity-resolution ablation study (string-only vs. transliteration-only vs. embedding-only vs. combined) being prepared as the research contribution for an IndiaAI Fellowship nomination — additive to the FYP timeline, not competing with it. Institute nomination eligibility to be confirmed with the guide.

4.4 ABDM Sandbox Integration — Medical History Pull (NEW)

What ABDM is: India's national digital health infrastructure — the Ayushman Bharat Health Account (ABHA) number links a patient's medical history for consent-based sharing across providers.

Scope — sandbox only, explicitly not production:

- Register on the ABDM developer sandbox (sandbox.abdm.gov.in).
- Build the consent-based Health Information User (HIU) flow: user links a sandbox ABHA number → grants consent → Kadi pulls relevant medical history into the case context, using the same schema as manually uploaded documents.
- Demo with ABDM's sandbox test patients — standard, legitimate practice for student/hackathon health-tech builds.

Explicitly out of scope: production ABDM certification. Going live requires clearing M1 → M2 → M3 milestones plus a CERT-IN/STQC-empanelled security audit — a 6-12 month process built for hospital-grade vendors, not achievable inside an FYP timeline. State this as a deliberate boundary in the final report, not a gap.

Why it's still worth building: a genuine, demoable integration with real government health infrastructure; strengthens the "linked, not co-located" pitch for Kadi; gives every module richer context without repeated uploads.

4.5 Auto-triggering

Once Kadi has enough context, it can proactively fire other modules without a re-upload. Bill mentions a ₹4L cardiac surgery under general category → Kadi checks SchemeSetu eligibility in the background and surfaces it alongside the BillNyay report. Strongest live-demo moment: one upload, several insights.

4.6 Consent boundary

User explicitly opts in per-case to letting SchemeSetu, DawaCheck, and (NEW) DaaviSetu see what BillNyay found, rather than data being pooled silently by default. This is also the strongest available answer if the data-handling question comes up in viva — lead with this over generic "we encrypt data" answers.

5. Cross-Module Technical Safeguards

(Consolidated from the challenges review — applies across all modules)

- **OCR accuracy:** confidence score shown; low confidence prompts a better image. **Additionally:** a manual correction step shows extracted fields to the user before the pipeline proceeds — turns "OCR might be wrong" into "OCR is a human-confirmed draft."
- **AI hallucinations:** RAG-grounded generation, plus mandatory source citation per claim (see §3.1) — unsupported claims are dropped, not softened.
- **Multilingual accuracy:** see §6.
- **Data freshness:** versioned reference datasets with visible "last updated" timestamps, recurring verification cadence per source (owned by Role 5), rate varying by how often each government source actually updates.
- **Privacy & security:** uploaded documents used for analysis only; concrete **data retention policy** to be finalized by the team (e.g., auto-delete case data after N days unless the user opts to retain); encryption at rest and in transit; Kadi's per-case consent boundary (§4.6) governs cross-module data flow.
- **Scalability:** realistic target stated explicitly as demo + pilot cohort (tens to low hundreds of concurrent users) — not production internet scale. If a scalability story is wanted for the report, basic load-testing (e.g., Locust) against ~50 concurrent users is honest and achievable; claiming "thousands" is not.

- **AI vs. professional advice:** positioned throughout as a decision-support system, not medical/legal advice. Disclaimer requires **active user acknowledgment** before appeal/claim/complaint generation — not passive footer text.
-

6. Multilingual Strategy — English, Hindi & Marathi (full scope)

Cross-cutting across all modules and Kadi — touches OCR, UI, LLM output generation, and entity resolution independently.

- OCR on Devanagari script needs its own validation pass — a separate technical risk from English OCR, needs a real Devanagari test set early.
 - **Source-vs-generation distinction** (still open, see §14): whether PMJAY/MJPJAY/CGHS/NPPA source documents exist natively in Hindi/Marathi determines whether "Marathi support" for that source means ingesting native-language source data or generating Marathi output from English source data — different engineering tasks, scoped per data source. (MJPJAY source material is confirmed natively Marathi, via jeevandayee.gov.in.)
 - **Terminology QA:** a native/fluent Hindi and Marathi speaker reviews generated appeal letters, claims, and scheme explanations for substantive correctness, not just fluency.
 - **Cross-script entity matching:** handled by Kadi's entity resolution pipeline (§4.3). Kept distinct from UI/output-language generation — related, but a different system.
 - **Shared deliverable:** the labeled trilingual medical-term test set needed for multilingual QA (Role 5) is the same dataset the Kadi entity-resolution ablation study needs — build once, serves both the FYP evaluation and the fellowship research note.
-

7. Technology Stack

Layer	Technology
Backend	FastAPI
Frontend	Next.js 15
Database	PostgreSQL + pgvector
Vector search	FAISS
LLM inference	Groq API
Transliteration	IndicXlit (AI4Bharat)
Cross-lingual embeddings	IndicSBERT (L3Cube)

Layer	Technology
Health record integration (NEW)	ABDM Sandbox APIs (sandbox.abdm.gov.in)
Containerisation	Docker Compose

Action item, still pending confirmation of completion: the original build used `llama3-70b` on Groq. Groq deprecated the 70B/8B Llama models on 17 June 2026. Re-point the pipeline to `openai/gpt-oss-120b` (or `qwen/qwen3.6-27b`). Verify against Groq's live deprecations page before committing, as availability shifts.

8. Data Sourcing

Module/Layer	Data needed	Format / risk
BillNyay	CGHS rate schedules (incl. city-tier/room-category variants)	Public PDF, English — scraping/parsing job
SchemeSetu	PMJAY (national) + MJPJAY (Maharashtra) eligibility rules, empanelled hospital lists	Public PDFs/circulars — confirm Hindi/Marathi availability per source
DawaCheck	NPPA Schedule-I ceiling price list (~800–900 formulations)	Public, likely English/structured — pull live at ingestion, count is volatile
DaaviSetu (NEW)	Blank claim/pre-authorization form templates, 2–3 common insurers to start	Public, published on insurer websites — not exhaustive for FYP scope
Kadi — ABDM (NEW)	Sandbox ABHA test accounts, HIU consent flow	Public developer sandbox, no production data
Kadi (entity resolution)	Labeled entity-pair dataset ("these two mentions are/aren't the same entity")	New requirement — folds into the existing self-donated test-document effort rather than a separate workflow

9. Naming Conventions

Element	Name
Project	ArogyaRakshak
Module 1	BillNyay

Element	Name
Module 2	SchemeSetu
Module 3	DawaCheck
Module 4 (NEW, unconfirmed)	DaaviSetu
Shared context layer	Kadi
Python package naming	lowercase, e.g. <code>billnyay</code> , <code>schemesetu</code> , <code>dawacheck</code> , <code>daavisetu</code> , <code>kadi</code>
DB table prefixes	<code>kadi_cases</code> , <code>kadi_entities</code> , <code>billnyay_cghs_rates</code> , <code>schemesetu_pmjay_rules</code> , <code>dawacheck_nppa_prices</code> , <code>daavisetu_claim_templates</code>
API route prefixes	<code>/api/billnyay/...</code> , <code>/api/schemesetu/...</code> , <code>/api/dawacheck/...</code> , <code>/api/daavisetu/...</code> , <code>/api/kadi/...</code>

10. Repository Structure

Monorepo — Kadi as shared infrastructure all modules depend on means separate repos would mean painful cross-repo dependency management for no real benefit at this team size.

```

arogyarakshak/
├── apps/
│   ├── web/           # Next.js 15 frontend
│   └── api/           # FastAPI backend
├── packages/
│   ├── kadi/          # shared context: extraction, entity resolution,
│   │               orchestration, ABDM integration
│   ├── billnyay/      # bill audit, appeal drafting, grievance tracking,
│   │               IRDAI complaint drafting
│   ├── schemesetu/
│   ├── dawacheck/
│   └── daavisetu/     # NEW - claim/pre-auth generation
├── data/              # CGHS / PMJAY / MJPJAY / NPPA / claim-template
│   ├── ingestion scripts + processed data
│   └── docs/          # this file and future documentation
└── docker-compose.yml

```

11. Team Roles

#	Role	Scope
1	Pipeline / orchestration	Multi-agent reasoning across all modules, Kadi's entity resolution pipeline, and (NEW) DaaviSetu's claim-drafting agent flow — the heaviest single role
2	OCR + data layer	Document ingestion, OCR incl. Devanagari validation, RAG ingestion for all reference datasets, (NEW) ABDM sandbox integration, (NEW) claim-form template structuring
3	Frontend / UX	Next.js interface, trilingual UI, (NEW) grievance-status tracker UI, disclaimer-acknowledgment flows
4	Infra / deployment	Docker, database, SSE streaming, hosting, (NEW) data retention policy implementation
5	Data + legal/eval + multilingual QA	Government data structuring, IRDAI/scheme rule accuracy, Hindi/Marathi terminology review, (NEW) monthly data-freshness verification cadence, (NEW) labeled trilingual test set — may need informal double-hatting given the expanded load

12. Feasibility Assessment

(New section — a straight technical/feasibility rating for every capability, old and new, for quick reference in scope discussions)

Capability	Feasibility	Notes
BillNyay bill audit + appeal drafting	✅ Buildable now	Carries forward existing hackathon build
SchemeSetu eligibility	✅ Buildable now	Public RAG sources
DawaCheck pricing check	✅ Buildable now	Scoped to Schedule-I only, by design
Kadi entity resolution	✅ Buildable now	Pretrained tools, no training pipeline needed
Document authenticity verification	❌ Not feasible, ever	No hospital/insurer database access — mitigated via integrity checks instead, not solved
DaaviSetu claim/pre-auth generation	✅ Buildable now	Form-fill only, no insurer API needed or available
Direct insurer claim submission	❌ Not feasible	No public insurer API exists
ABDM medical history pull	✅ Buildable — sandbox only	Production certification (6-12mo, security audit) is out of scope
Grievance status self-tracking	✅ Buildable now	Self-reported, no external API needed
Automated Bima Bharosa status pull	❌ Not feasible	No public API; login-gated portal
Auto-drafted IRDAI complaint	✅ Buildable now	User submits themselves — required by IRDAI's own policy
Automated IRDAI submission on user's behalf	❌ Not feasible / against policy	IRDAI explicitly does not accept third-party/agent-filed complaints
Production-scale (1000s of concurrent users)	❌ Not the target	FYP target is demo + pilot cohort; state this explicitly rather than overclaiming

13. Decisions Resolved

✅ BillGuard renamed to BillNyay
 ✅ Cross-module linking architecture designed — Kadi, shared case context, not a fourth module
 ✅ Entity resolution: full version approved, cross-script included, built on IndicXlit + IndicSBERT rather than from scratch
 ✅ Repo structure: monorepo, structure above
 ✅ (NEW) Claim/pre-auth generation, ABDM

sandbox integration, grievance self-tracking, and auto-drafted IRDAI complaints added to scope, all reframed to stay within real technical and policy constraints ✅ (NEW) All 16 challenges-review fixes adopted as committed scope, not optional polish

14. Open Items — Still Pending

Not decidable from a planning conversation alone — need the team to go find out:

- Confirm with guide whether hackathon-built BillNyay code can carry into the FYP submission directly, or must be framed as new work per department rules
 - Confirm SPPU Phase-I review date and what "done" needs to look like by then vs. final submission
 - Resolve Hindi/Marathi source-vs-generation per data source (§6) — determines real workload
 - Execute the Groq model swap (§7) — flagged, not yet confirmed done
 - Decide synthetic vs. real self-donated test-document dataset for OCR validation
 - Plan the labeled entity-pair dataset needed to tune entity-resolution confidence thresholds (§8), shared with the multilingual QA test set (§6)
 - Finalize the concrete data retention window (N days) for uploaded documents (§5)
 - (NEW) Confirm claim form field structures for 2-3 common insurers — not an exhaustive list, for FYP scope
 - (NEW) Decide DaaviSetu naming/branding, or fold it into BillNyay as a sub-flow instead of a separate module
 - (NEW) Register for ABDM sandbox access and complete an initial HIU flow walkthrough
 - (NEW) Confirm institute nomination eligibility for the IndiaAI Fellowship with the guide
-

15. Explicitly Out of Scope

- Non-Schedule-I drug price variance — no fixed government ground truth to benchmark against
- State schemes beyond Maharashtra — architecture designed to extend, not built for this submission
- Any held or stored patient data beyond the defined retention window — every module stays bring-your-own-document
- UI/output-language generation being solved by the same tooling as entity resolution — related systems, kept separate (§6)
- Document authenticity/forgery verification — structurally impossible without hospital/insurer database access

- Direct insurer API integration for claim submission — no such public API exists
 - Production ABDM certification (M1-M3, security audit)
 - Automated submission to Bima Bharosa/IRDAI on the user's behalf — against IRDAI's stated third-party policy
 - Automated status-polling of Bima Bharosa — no public API; login-gated
 - Production-internet-scale load handling — target is demo + pilot cohort
-

16. Status

Architecture, naming, and full module set (including new capabilities) scoped and reframed for technical/policy feasibility as of this document. Next step: implementation, per the role split in §11, with the feasibility ratings in §12 as the guardrail against scope drift.